

Phase 0 — Architecture & Guardrails

Goal: Lock scope and non-negotiables.

Define:

- GitHub is the **source of truth**
- App is a **read-only intelligence layer**
- AI must always cite GitHub URLs
- No manual editing of repo summaries in DB
- Custom Properties are required for all repos

Design decisions:

- Ingestion model: Webhook + periodic reconciliation
- RAG model: Hybrid search (PGVector + keyword)
- Token budget rules
- Indexing granularity (file-level, not whole repo blob)

Deliverables:

- System architecture diagram
- Data model draft
- Ingestion spec
- AI prompt strategy doc

This prevents chaos later.

Phase 1 — GitHub Integration Layer

(Sequential First, then Parallel)

Stack: GitHub API + QStash + SvelteKit API routes

Build:

- GitHub App (org-installed)
- Webhook ingestion endpoint
- Repo sync worker
- Scheduled reconciliation job

Data pulled:

- Repos
- Default branch
- README
- [/docs](#) directory
- CODEOWNERS
- Custom properties
- Language stats
- Last commit metadata

Store raw content in Postgres.

Parallelizable once base webhook works.

Phase 2 — Data Model & Storage Foundation (Parallel)

Neon Postgres + pgvector

Core tables:

- organizations
- repositories
- repo_metadata
- repo_custom_properties
- repo_files
- repo_docs
- repo_embeddings
- repo_summaries
- ingestion_jobs
- sync_logs

Index:

- File-level embeddings
- README + architecture docs separately
- Optional: issue/discussion embeddings later

Design for:

- Incremental updates
 - Deleting stale embeddings
 - Multi-org capability
-

Phase 3 — Embedding & RAG Pipeline (Parallel)

Stack: OpenRouter + PGVector

Build:

- Chunking strategy (1000–1500 tokens, overlap 150)
- Embedding pipeline
- Hybrid search (vector + ILIKE fallback)
- Context assembler
- Citation generator (line-level GitHub permalinks)

AI behaviors:

- Repo overview generation
- Architecture summary generation
- “Where is X implemented?” Q&A
- “How do I run this?” extraction

All responses must:

- Include GitHub links
 - Include file paths
 - Show confidence level
-

Phase 4 — Repo Intelligence Engine (Parallel)

This is the system’s differentiator.

Features:

- Auto-generated:
 - Problem summary
 - Tech stack detection
 - Entry points
 - Architectural style classification
 - Dependency highlights

- Risk detection:
 - No README
 - No CODEOWNERS
 - Deprecated repo
 - Inactive > X months

Tech stack inference logic:

- package.json
- requirements.txt
- pom.xml
- Dockerfile
- etc.

This layer uses:

- Heuristics first
 - AI second
-

Phase 5 — Internal Web App (SvelteKit UI) (Parallel)

UI modules:

1) Repo Catalog

- Filters by:
 - owner_team
 - lifecycle
 - tech stack
 - domain
- Health indicators

2) Repo Detail Page

- Human-authored summary
- AI summary
- Architecture section
- Tech stack card
- Key files
- Code reference links
- Ask-this-repo chat panel

3) Org Chat

- “Where do we handle payments?”
- “Which services use Redis?”
- “What repos are inactive?”

4) Audit Mode

- Identify stale repos
- Identify missing documentation
- Identify duplication

Phase 6 — Automation & Governance Layer (Parallel)

- PR check:
 - Enforce README sections
 - Enforce docs/architecture.md
 - Enforce custom properties
- Scheduled nightly:
 - Reconcile repo list
 - Detect deleted repos
- Slack notifications:
 - Missing documentation
 - Repo stale warnings
 - Failed ingestion

This is how you prevent entropy.

Phase 7 — Intelligence Scaling & Optimization (Later)

Add:

- Cross-repo dependency graph
- Service-to-service inference
- Repo similarity detection
- Duplicate solution detection
- “Before building X, check these repos” suggestion engine
- Org-wide architecture map

This should not block MVP.

Product Requirements Document (PRD)

You can give this to your AI agent.

Product Requirements Document

1. Overview

RepoIntel is an internal AI-powered enterprise web application that provides high-level, searchable, and structured intelligence across all GitHub repositories within an organization.

The system indexes repository metadata, documentation, and source code to provide:

- Clear repository context
- Architecture summaries
- Tech stack detection
- Code reference discovery
- Cross-repo search

- Audit visibility

GitHub remains the source of truth.

2. Goals

Primary Goals

1. Reduce time to understand a repository by 70%
2. Improve discoverability of past work
3. Prevent duplicate implementations
4. Enable structured auditing of repo health
5. Provide AI Q&A grounded in GitHub content

Non-Goals

- Replace GitHub
 - Replace code review
 - Provide fully autonomous architecture inference
 - Allow editing repo content inside the portal
-

3. Tech Stack

Frontend:

- Svelte + SvelteKit
- Deployed on Vercel

Backend:

- SvelteKit API routes
- Neon Postgres
- PGVector

Async:

- QStash

AI:

- OpenRouter (LLMs + embeddings)

Integration:

- GitHub REST + GraphQL APIs
 - GitHub Webhooks
 - GitHub App (Org-installed)
-

4. Core Functional Requirements

4.1 GitHub Sync

System must:

- Sync all org repos
 - Store metadata
 - Fetch README
 - Fetch /docs folder
 - Extract CODEOWNERS
 - Store custom properties
 - Update on webhook trigger
 - Nightly reconcile
-

4.2 Embedding & Retrieval

System must:

- Chunk content at file level
 - Store embeddings in PGVector
 - Support hybrid search
 - Return citations with permalinks
-

4.3 AI Q&A

System must:

- Support repo-scoped Q&A
- Support org-wide Q&A
- Cite sources
- Provide confidence rating
- Limit hallucination via retrieval gating

4.4 Repo Overview Page

Must display:

- Repo name
- Owner team
- Lifecycle
- Last updated
- Languages
- Tech stack
- Problem solved summary
- Architecture summary
- Key files
- Documentation completeness score
- Risk flags

4.5 Governance & Health

System must detect:

- Missing README
- Missing architecture doc
- No CODEOWNERS
- Inactive repo > 6 months
- Archived repos
- Duplicate similar repos

5. Data Model (High-Level)

repositories
repo_metadata
repo_custom_properties
repo_files
repo_embeddings
repo_summaries
repo_health_scores
sync_jobs
audit_flags

6. Security Requirements

- Private org-only access
 - GitHub App permissions least privilege
 - All AI calls logged
 - No source code leaves system except to model provider
 - Option to restrict AI models to approved providers
-

7. Performance Requirements

- Repo page loads < 1.5s
 - Org search < 2s
 - AI response < 6s average
 - Sync incremental, not full reindex
-

8. Metrics for Success

- Time to understand repo reduced
 - % repos with required docs
 - AI answer citation rate
 - Reduction in duplicate repos
 - Monthly active usage
-

9. Phased Delivery

MVP:

- Repo sync
- Repo page
- AI per-repo Q&A
- Custom property filtering

V2:

- Org-wide chat

- Audit dashboard
- Repo similarity detection

V3:

- Cross-repo architecture inference
 - Suggest-before-build intelligence
-

Critical Advice

Here's where most teams fail:

- They overbuild AI before enforcing documentation standards.
- They let AI generate authoritative summaries without review.
- They don't enforce custom properties.
- They don't build governance checks.

If you want this to succeed:

Standardize first.

Index second.

Add AI third.